



**Universidad
Europea** MADRID

Informe Ejecutivo de Seguridad

Pipeline DevSecOps sobre la aplicacion euVWA

Oussama Marzak Faroit

Desarrollo Seguro de Web y Apps

Actividad 2. Caso de uso practico SecDevOps

Universidad Europea de Madrid

Mayo de 2026

Indice

1. Introduccion y contexto
2. Alcance
3. Riesgos identificados
4. Controles de seguridad implementados
5. Resultados de la evaluacion
6. Conclusiones y recomendaciones

1. Introduccion y contexto

euVWA es una aplicacion web inspirada en DVWA y desarrollada en Node.js con Express, que reproduce vulnerabilidades habituales del OWASP Top 10 para poder estudiarlas y compararlas con su version corregida. Esta actividad parte de esa aplicacion y construye sobre ella un pipeline completo de DevSecOps.

El objetivo es integrar la seguridad de forma nativa y automatizada en todo el ciclo de vida del software, siguiendo el enfoque Shift Left, de modo que las vulnerabilidades se detecten y se bloqueen lo antes posible, antes de llegar a un despliegue.

2. Alcance

El trabajo se organiza en dos ramas paralelas con las mismas funcionalidades:

- Rama main-vulnerable: contiene las vulnerabilidades intencionadas y una dependencia con CVE conocido. Sirve para demostrar que el pipeline detecta y bloquea.
- Rama main-secure: implementa las mismas funcionalidades aplicando buenas practicas. Sirve para demostrar que el pipeline aprueba un codigo seguro y publica el artefacto.

El pipeline se ejecuta en GitHub Actions e incorpora analisis estatico, analisis de dependencias, inventario de software, construccion y escaneo de la imagen, analisis dinamico y publicacion en GitHub Container Registry.

3. Riesgos identificados

Los principales riesgos que el pipeline debe controlar son los siguientes:

- Vulnerabilidades de codigo del OWASP Top 10: inyeccion SQL, XSS reflejado y almacenado, inyeccion de comandos, subida insegura de archivos, autenticacion debil, control de acceso roto, mala configuracion de seguridad y exposicion de datos sensibles.
- Dependencias con vulnerabilidades conocidas, capaces de comprometer la aplicacion aunque el codigo propio sea correcto.
- Vulnerabilidades heredadas de la imagen base del contenedor.
- Secretos incrustados en el codigo y cabeceras de seguridad ausentes.

4. Controles de seguridad implementados

- SAST con Semgrep y ESLint: analiza el codigo fuente y bloquea ante patrones inseguros como la inyeccion de comandos, la inyeccion SQL por concatenacion o los secretos incrustados.
- Analisis de dependencias y SBOM con Trivy: detecta librerias vulnerables y genera el inventario de componentes en formato CycloneDX.
- Construccion de una imagen Docker endurecida: multi-stage, usuario sin privilegios, imagen minima, sin npm en ejecucion y sin secretos.
- Escaneo de la imagen con Trivy: revisa las vulnerabilidades de la imagen final.
- DAST con OWASP ZAP: analiza la aplicacion en ejecucion y detecta cabeceras y cookies inseguras.

- Publicacion condicionada en GHCR: la imagen solo se publica si todos los controles anteriores pasan.

5. Resultados de la evaluacion

La siguiente tabla resume el resultado real de la ejecucion del pipeline en cada rama:

Control	Rama vulnerable	Rama segura
SAST (Semgrep y ESLint)	Falla: SQLi, secreto en codigo e inyeccion de comandos	Pasa sin hallazgos relevantes
Dependencias (Trivy)	Falla: 12 vulnerabilidades (lodash y multer)	Pasa: 0 vulnerabilidades
Imagen Docker (Trivy)	Falla: 1 critica y 11 altas	Pasa: 0 vulnerabilidades
DAST (OWASP ZAP)	14 alertas (3 medias, 6 bajas)	10 alertas (3 medias), 0 altas
Publicacion en GHCR	No se ejecuta	Publica la imagen
Resultado global	Falla y bloquea el despliegue	Pasa y publica la imagen

La rama vulnerable se bloquea en el analisis estatico, en el analisis de dependencias y en el escaneo de la imagen, por lo que no llega a publicar ningun artefacto. La rama segura supera todos los controles y publica la imagen en el registro de contenedores.

6. Conclusiones y recomendaciones

El pipeline cumple el objetivo de integrar la seguridad de forma automatizada y demuestra el enfoque Shift Left: el codigo inseguro se detiene antes del despliegue y solo el codigo seguro genera un artefacto publicable. El contraste entre las dos ramas evidencia el valor de los controles.

Como lineas de mejora se proponen las siguientes:

- Anadir un analisis dinamico en modo activo, ademas del baseline, para detectar inyecciones explotables en tiempo de ejecucion.
- Firmar la imagen publicada y verificar la firma antes del despliegue.
- Fijar por digest las imagenes base y las versiones de las acciones.
- Incorporar pruebas automatizadas y revisar de forma periodica las excepciones aceptadas en el analisis de dependencias.